CLICK-OPS

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

Overview

SETUP AZURE

This phase covers configuring Microsoft Entra ID (formerly Azure Active Directory) as the identity provider for an employee portal application, including app registration, token configuration, and the creation of a role-based access control (RBAC) system using native Azure AD security groups.

FIND APP REGISTRATION

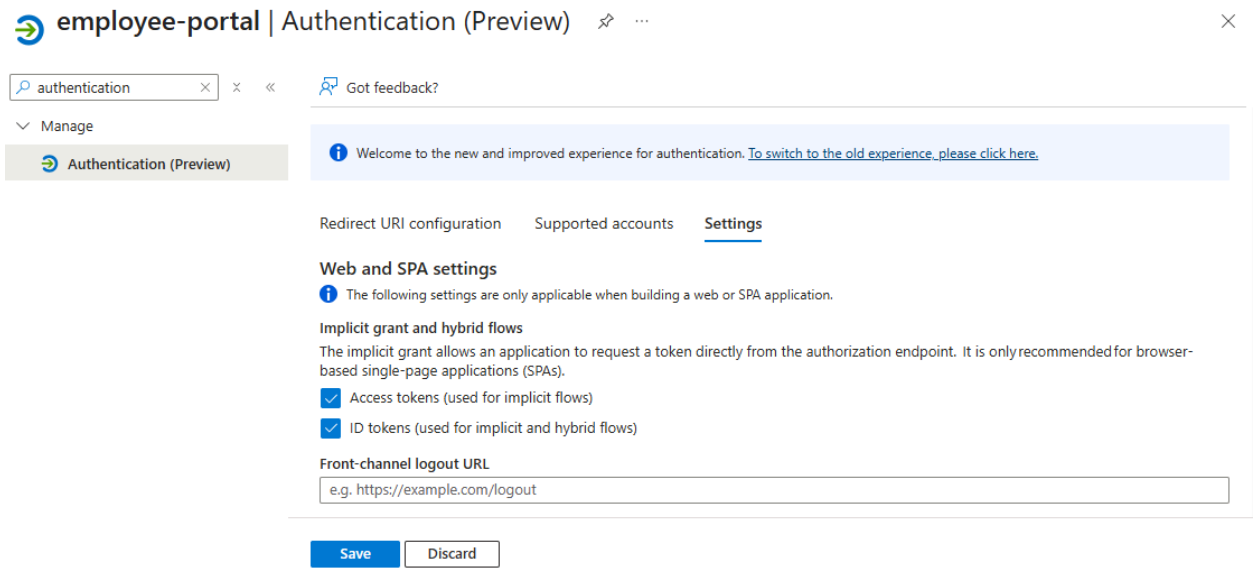
- In the Azure portal, search “app registration
- Register the app.
- Name: **employee-portal**
- Supported account types: single tenant, my organization only
- Redirect URI:
    1. Platform: single-page application SPA
    2. URL: http://localhost:3000
- Register: save your IDs. Application (client DI) and directory (tenant ID). Copy these for it will be needed later
- On the left: API permissions
- Left sidebar → API permissions
- Click "+ Add a permission"
- Choose Microsoft Graph
- Choose Delegated permissions
- Search and add: UserRead
- Click "Grant admin consent" → Yes

ENABLE TOKEN

- Left sidebar → Authentication

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

- Scroll down to Implicit grant and hybrid flows
- Check ID tokens
- Check Access tokens
- Click Save
- 

CLIENT SECRET

Left sidebar → Certificates & secrets
Click "+ New client secret"
Description: employee-portal-secret
Expires: 24 months
Click Add
Copy the Value immediately — it only shows once

RBAC

In here, we will be creating groups and assigning users to that group to demonstrate RBAC (role-based access control)

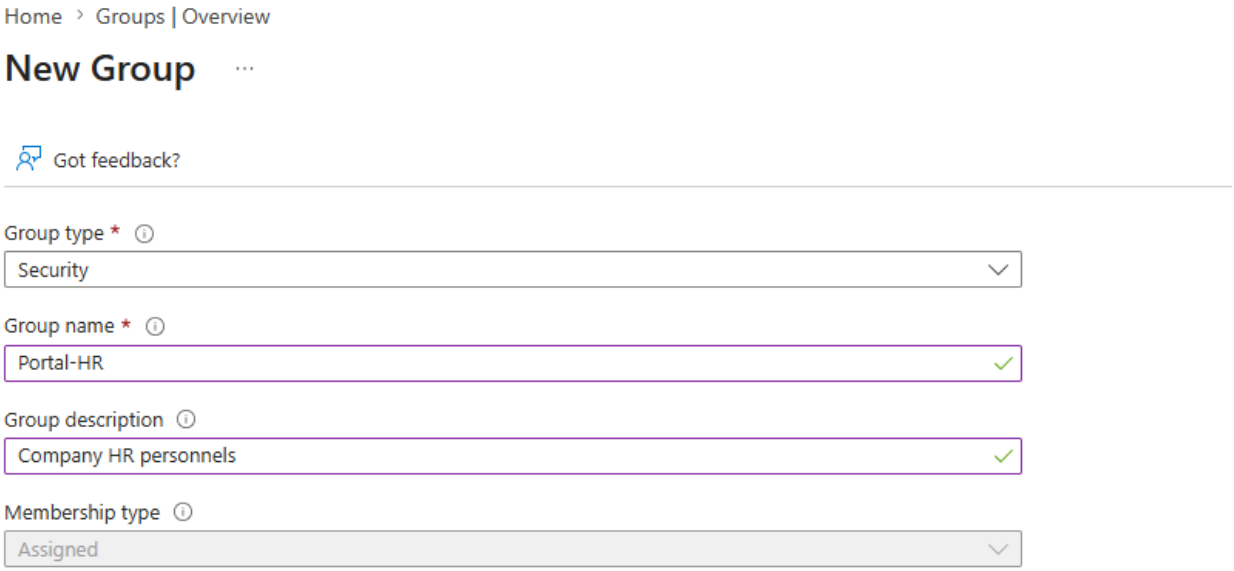
Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

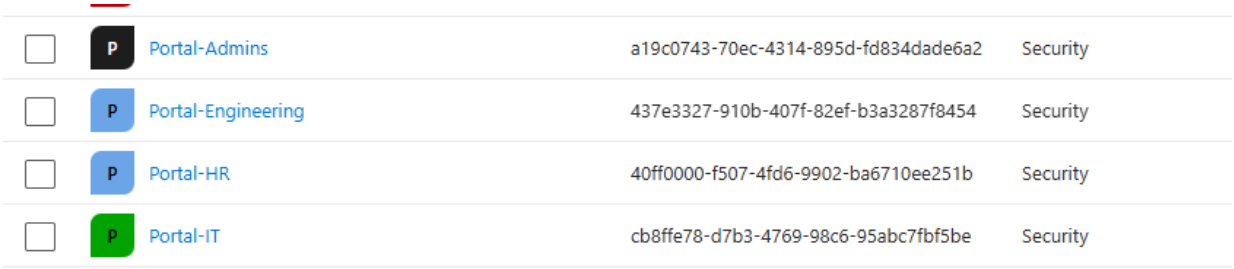
Step 1 - Create the Groups

-

Group Name	Group Type	Membership type
Portal-HR	security	Assigned
Portal-IT	security	Assigned
Portal-Engineering	security	Assigned
Portal-Admins	security	Assigned



-



-
- ***For each created group name, note down the "Object ID" since you will need these 4 IDs later to map them in React.***

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

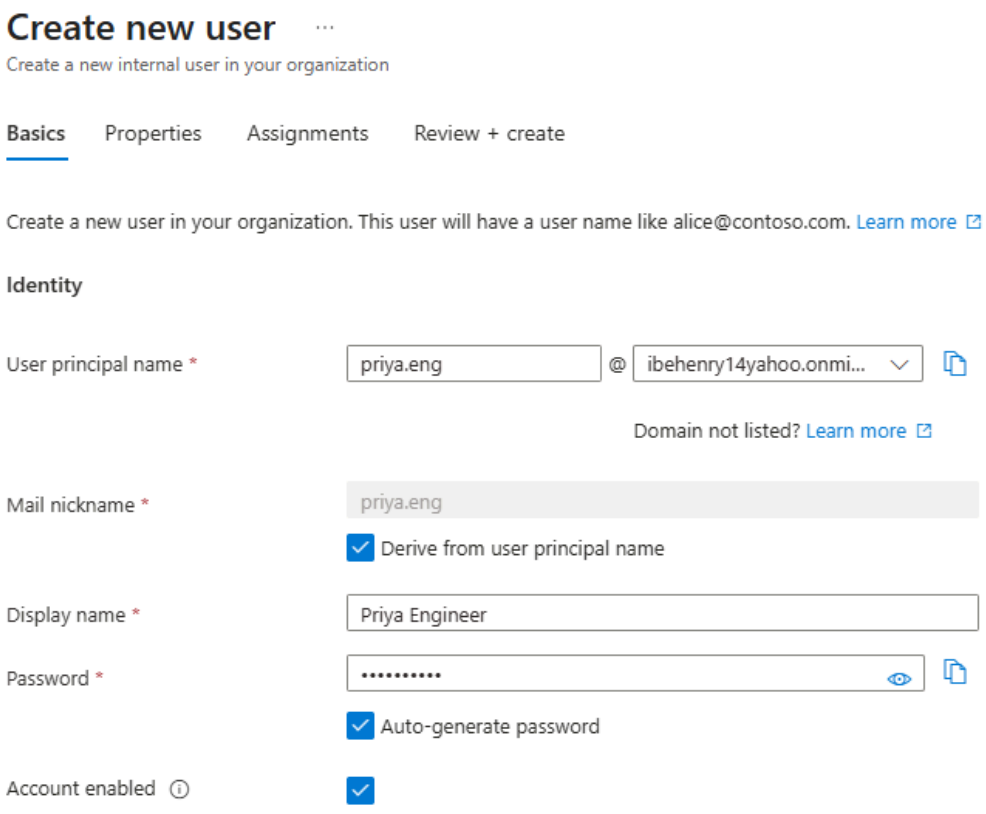
AUTHOR : HENRY IBE

Step 2 - Create Test Users

-

Display Name	User Principal Name
Sarah HR	sarah.hr@yourtenant.onmicrosoft.com
Marcus IT	marcus.it@yourtenant.onmicrosoft.com
Priya Engineer	priya.eng@yourtenant.onmicrosoft.com
Admin User	admin.user@yourtenant.onmicrosoft.com

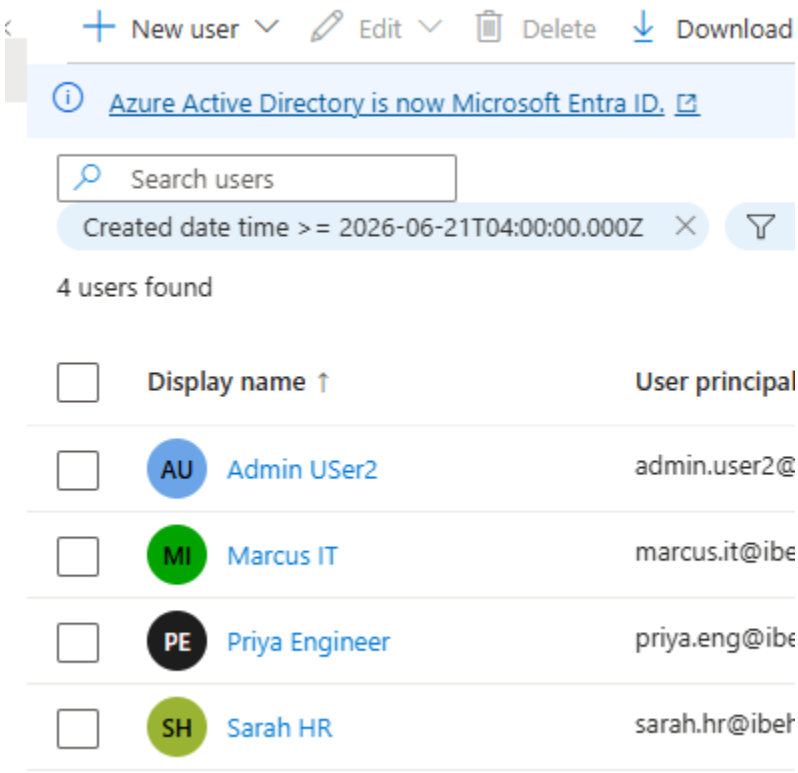
- ***Note: yuou might need to reset password after first login***



-

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE



-
- ***Note: Above is the image of the created users.***
-

Step 3 - Add Each User to their matching groups

In the Microsoft Entra ID, under each group, +add members

-

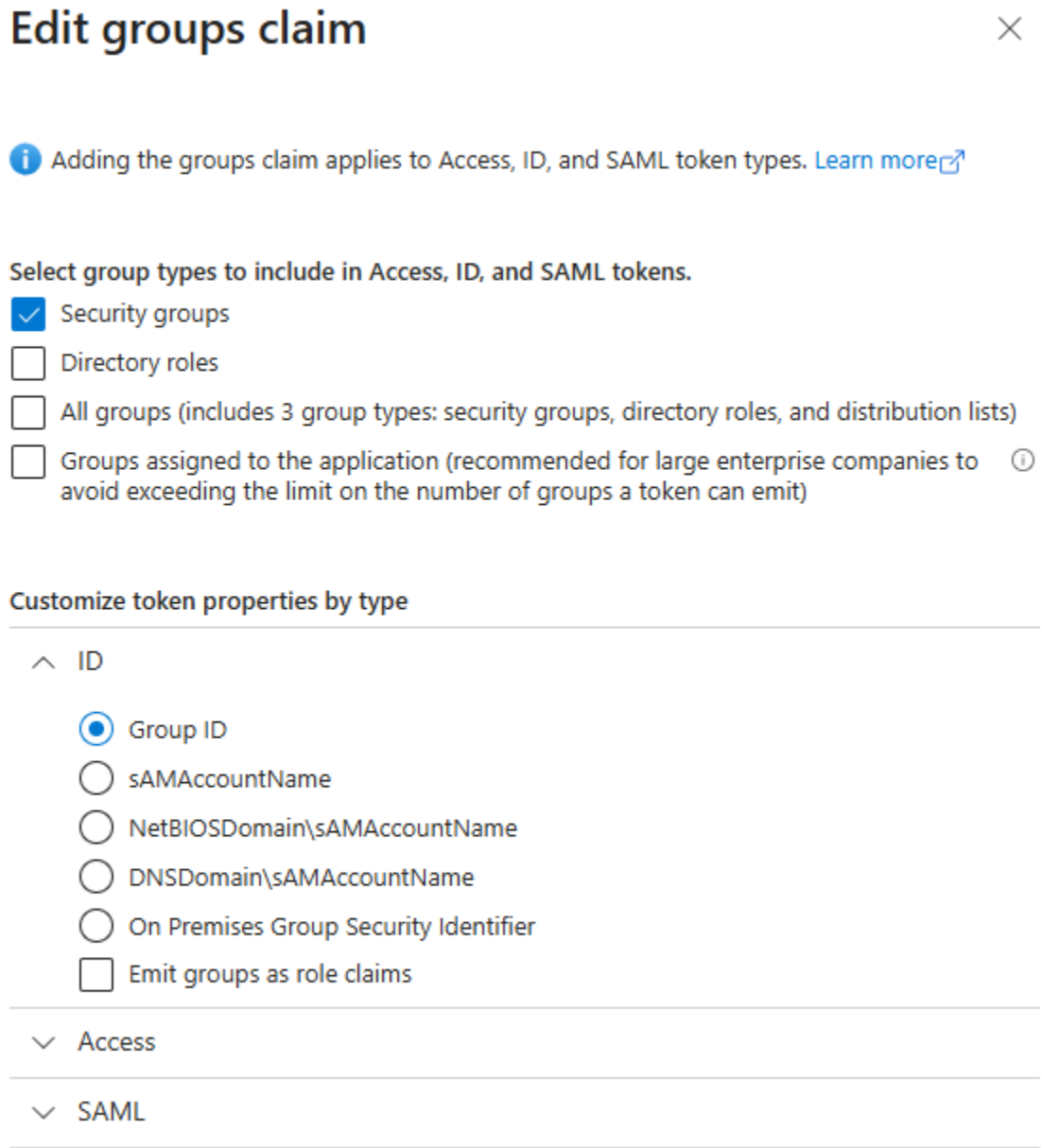
Group	Add this user
Portal - HR	Sarah HR
Portal - IT	Marcus IT
portal-engineering	Priya Engineer

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

Portal-Admins	Admin User2

- After mapping each user to their respective group. Copy the object ID of each group. This will be used in our React app
- *In the App Registration's Token configuration, a groups claim was added and scoped to security Groups, with the ID token configured to emit each group's object ID.*



-

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

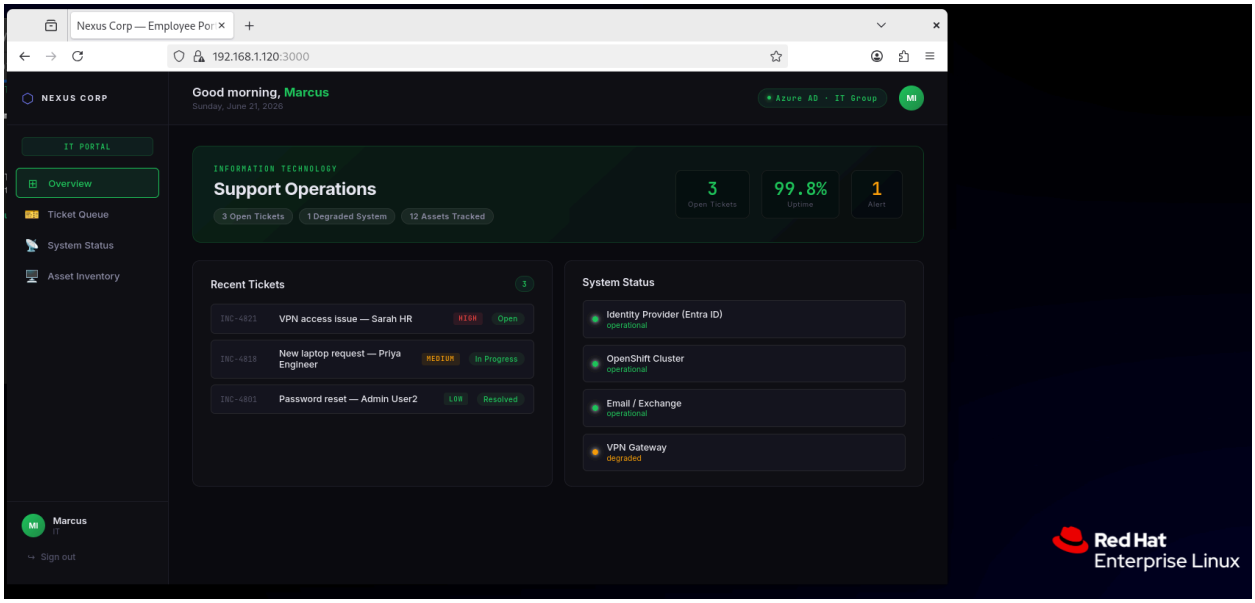
AUTHOR : HENRY IBE

- With the groups claims enabled, every time a user authenticates, Azure issues an ID token that already contains the GUIDs of every security group they belong to; no extra API call is needed to determine their role. The application can decode this token client-side and make routing decisions (which dashboard or webpage to render based on claims it can trust, since they were issued and signed by Azure AD itself. This reflects a real enterprise authorization pattern: identity and group membership are managed centrally in the identity provider, and applications consume that as a signed source of truth rather than maintaining their own separate roles database.

Sample Login

Here we will be logging in with one of the users and see what web page is shown to us when we log in.

- Below, we logged in with user “Marcus IT,” which is assigned to the IT department, and we can see it routes directly to the IT webpage



Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

PHASE 2- The web App

Overview

With Entra ID configured as the identity provider, Phase 2 covers the React application that consumes that identity layer: authenticating users via MSAL, decoding their token claims, and routing each employee to a department-specific dashboard based on their Azure AD group membership.

Authentication Flow

The app uses **MSAL.js** (@azure/msal-browser and @azure/msal-react) configured as a Single-Page Application, matching the SPA platform registered in Entra ID during Phase 1.

When an unauthenticated user lands on the app, they see a login gate with a single "Sign in with Microsoft" action. This triggers a redirect to Microsoft's actual hosted login page — the application never sees or handles credentials directly. After successful authentication, Azure redirects back to the app with a signed ID token attached to the user's MSAL account session.

Reading Token as a Source of Truth

Once authenticated, MSAL exposes the decoded ID token claims on the user's account object. Rather than calling an external API to determine the user's role, the application reads the groups claim directly from this token — the same claim configured in Phase 1's Token Configuration step.

This is the core architectural decision of the project: **authorization data travels inside the authentication token itself.** The application doesn't ask "who is allowed to see what" — it asks "what does this signed token say," and trusts the answer because it trusts Azure AD as the issuer. There is no local roles table, no separate permissions database, and no additional round-trip required to determine access.

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

Route Guard Pattern

A dedicated route guard component sits between authentication and the dashboard views. Its job is narrow and deliberate:

1. Read the groups array from the token claims
2. Map each group's Object ID to a department label, using the mapping established in Phase 1
3. Render the dashboard that matches the user's department
4. If no group matches, render an **Access Restricted** view instead of a blank page or an error

That fourth behavior is the most important one architecturally. A system that silently fails or crashes when access is denied is harder to trust than one that explicitly and gracefully says "you don't have access, and here's why." The Access Restricted view confirms the user authenticated successfully, but makes clear that authentication and authorization are two separate concerns — being a verified employee doesn't automatically grant access to every part of the system.

When a user belongs to multiple groups (for example, an admin who is also tagged in HR), the route guard applies a priority order so the most privileged view wins, rather than producing ambiguous or unpredictable routing.

Department Dashboards as a Shared Shell

Rather than building four entirely separate applications, each department dashboard (HR, IT, Engineering, Admin) is built on top of one shared layout shell — consistent sidebar, header, and navigation structure — with department-specific color identity, content, and data underneath.

This keeps the codebase maintainable while still making each department's experience visually and functionally distinct on screen: HR's pink-accented workforce view looks and feels nothing like IT's green-accented ticket queue, even though both are powered by the same underlying shell component receiving different configuration.

The Admin dashboard is treated as a superset view — rather than managing one department, it surfaces a summary across all of them, reflecting the elevated nature of that group's access.

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

What This Demonstrates

End to end, the application proves a real authorization pattern: a user authenticates once against a central identity provider, and everything the application shows them afterward is derived from claims that provider issued and signed. Different users, using the same login flow against the same application, land on completely different experiences — automatically, with no manual configuration per user inside the app itself.

Scope note, consistent with Phase 1: this authorization check happens at the application layer, inside the React client. A production system handling sensitive data would pair this with server-side validation of the same token and claims before returning any protected data — the frontend route guard improves user experience and prevents accidental access, but a backend service would be the actual security boundary in a system handling regulated data.

PHASE 3- Docker Build & OpenShift Deployment

Step 1 - Docker build

Download Docker if you do not have one yet

```
sudo dnf install podman-docker -y

Docker - - version
```

```
cd ~/Azure-EntraID-OpenShift/employee-portal
docker build -t employee-portal:latest .

************************************************************
This image was built in two stages
    ● Builder stage: pulled node:18-alpine
    ● Installed dependencies
    ● Ran “ npm run build
```

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

- Generated React production files in /app/build

Runtime stage
- Pulled nginx:alpine
- Copied React build artifacts into /usr/share/nginx/hmtl
- Configured nginx
- Switched to non-root user:
- Exposed port 8080
- Image was tagged as employee-portal:latest

Let's run it locally to test

```
docker run -p 8080:8080 \
  -e REACT_APP_AZURE_CLIENT_ID=cb688456-eedf-404b-8387-a13d23ab568a \
  -e REACT_APP_AZURE_TENANT_ID=0efb94ec-ff0d-4bb4-95cd-dd0e911b3138 \
  -e REACT_APP_REDIRECT_URI=http://localhost:8080 \
  employee-portal:latest
```

Step 2: Tag the Image with your Docker Hub Username

Create the repo manually on Docker Hub
- Create repo
- Name: employee-portal
- Visibility: public
- Create
-

```
docker tag employee-portal:latest [docker_username]/employee-portal:latest
```

******

Push It

```
docker push [docker_username]/employee-portal:latest
```

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

Step 3: OpenShift Deployment

Confirm you have access to the OpenShift cluster (CRC locally, or developer sandbox)

Crc status

```
CRC VM:          Running
OpenShift:       Running (v4.21.8)
RAM Usage:       11.12GB of 20.96GB
Disk Usage:      43.1GB of 63.82GB (Inside the CRC VM)
Cache Usage:     31.52GB
```

******Navigate your project and view the YAML*****

cd ~/Azure-EntraID-OpenShift/employee-portal
cat openshift-deploy.yaml

Here we will edit the openshift-deploy.yaml file

Make these exact replacements

Find	Replace with
YOUR_TENANT_ID	(your tenant ID from Entra ID)
YOUR_CLUSTER_DOMAIN	apps-crc.testing
YOUR_CLIENT_ID	(client ID from Entra ID)
YOUR_REGISTRY	docker.(username)
YOUR_CLIENT_SECRET	From Azure Entra ID

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

** After editing the openshift-deploy.yaml file with the right values****

- oc apply -f openshift-deploy.yaml

```
project.project.openshift.io/employee-portal created
configmap/portal-config created
secret/portal-secret created
deployment.apps/employee-portal created
service/employee-portal created
route.route.openshift.io/employee-portal created
```

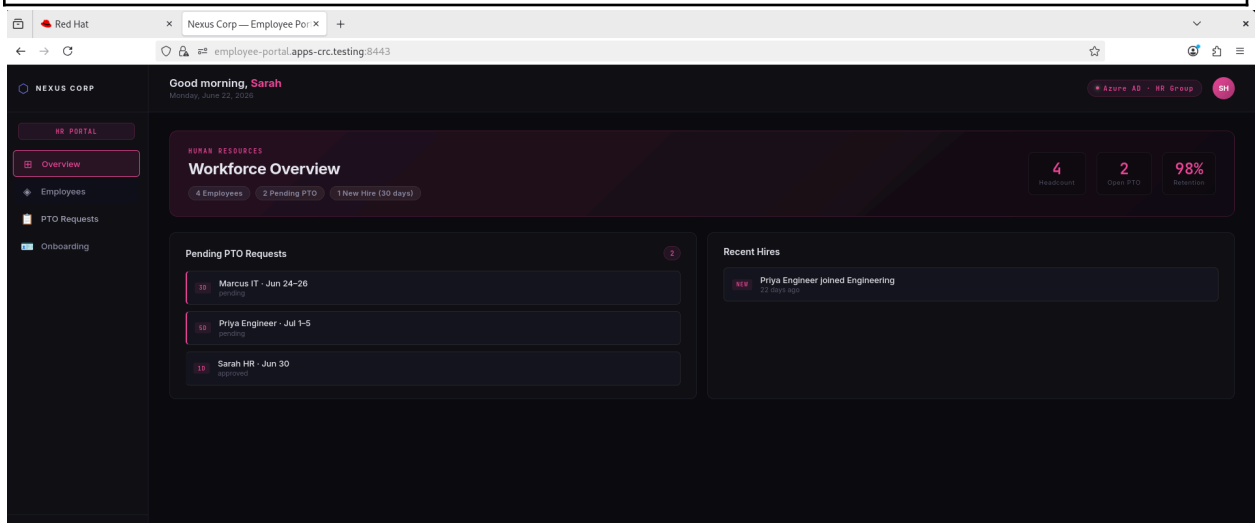
- Oc get pods -n employee-portal

```
NAME                                READY   STATUS    RESTARTS   AGE
employee-portal-7b445964bf-9m584    1/1     Running   0          3m51s
```

*** Now let's get the URL and hit it***

oc get route employee-portal -n employee-portal

```
NAME              HOST/PORT                          PATH   SERVICES          PORT   TERMINATION     WILD
CARD
employee-portal   employee-portal.apps-crc.testing          employee-portal   8080   edge/Redirect   None
```



Step 4 — Resolving Network Access to the Route

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

After applying the manifest and confirming the pod reached Running status, the OpenShift Route returned employee-portal.apps-crc.testing as the external hostname. Hitting this URL directly, however, surfaced a layered networking challenge worth documenting, since it reflects a realistic debugging scenario rather than a clean "it just worked" deployment.

The issue: CRC's router (apps-crc.testing ingress) binds only to the loopback interface (127.0.0.1) on the host machine running the CRC VM — not to the machine's external network IP. This is CRC's default, local-only design. Since the browser accessing the application was running on a separate physical machine (connected via SSH to the host running CRC), a direct request to the host's LAN IP on port 443 was refused, even though the hostname resolved correctly via /etc/hosts.

The fix: an SSH local port-forward tunnel was established from the client machine to the CRC host:

ssh -L 8443:127.0.0.1:443 -L 8080:127.0.0.1:80 <user>@<crc-host-ip>

This forwards local port 8443 on the client machine through the SSH connection directly to port 443 on the CRC host's loopback interface — exactly where the OpenShift router is actually listening. The corresponding /etc/hosts entry on the client machine was set to resolve employee-portal.apps-crc.testing to 127.0.0.1, so that traffic would route through the tunnel rather than attempting a direct LAN connection.

This is a common pattern when working with local development clusters (CRC, Minikube, kind) from a separate client machine, and it mirrors a deliberate security default — local clusters aren't designed to expose ingress to a network by default, and tunneling is the expected way to reach them remotely without changing that posture.

Step 5 — Aligning the Build-Time Redirect URI

A second issue surfaced once the Route was reachable: after a successful Microsoft sign-in, Azure correctly redirected back to the application — but to the wrong host. The browser was sent to the original local development address ([redacted]) rather than the OpenShift Route's HTTPS URL.

Root cause: Create React App embeds all REACT_APP_* environment variables into the static JavaScript bundle at **build time**, not at container runtime. The OpenShift ConfigMap and Secret correctly inject environment variables into the running container's process environment — but by that point, the React build has already happened, and the redirect URI value used during npm

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

run build is permanently baked into the compiled bundle. Passing different values via oc apply, docker run -e, or the ConfigMap does not affect a build that has already completed.

The fix: the .env file was updated to the exact Route URL (including the tunneled port), and the image was rebuilt, re-tagged, and re-pushed:

```
**** vim .env**** editing the .env and adding this below

REACT_APP_REDIRECT_URI=https://employee-portal.apps-crc.testing:8443


docker build -t employee-portal:latest .
docker tag employee-portal:latest <dockerhub-user>/employee-portal:latest
docker push <dockerhub-user>/employee-portal:latest
oc rollout restart deployment employee-portal -n employee-portal
```

The corresponding redirect URI was also registered as an additional **Single-page application** entry in the Entra ID App Registration's Authentication settings, since Azure strictly validates that the redirect URI returned in the request matches one explicitly configured for the application — a deliberate anti-spoofing control, not an arbitrary restriction.

Architectural takeaway: for client-side applications using build-time environment injection (as opposed to a server-rendered app reading process.env at request time), any environment-specific configuration — redirect URIs, API base URLs, tenant IDs — must be correct *before* the image is built. ConfigMaps and Secrets remain valuable for runtime configuration consumed by a backend process, but a static frontend bundle has already "frozen" those values by the time Kubernetes ever sees the container.

Result

With both issues resolved, the application was reachable end-to-end through the OpenShift Route, and the full authentication and RBAC flow — established in Phases 1 and 2 — operated correctly against the containerized, cluster-hosted deployment: different test users, signing in through the same Route URL, were routed to their respective department dashboards exactly as they had been in local development, with one difference — the application was now running entirely inside an OpenShift pod rather than the developer's local Node process.

Enterprise Identity-as-Code — Azure Entra ID, OpenShift, Terraform & GitOps

AUTHOR : HENRY IBE

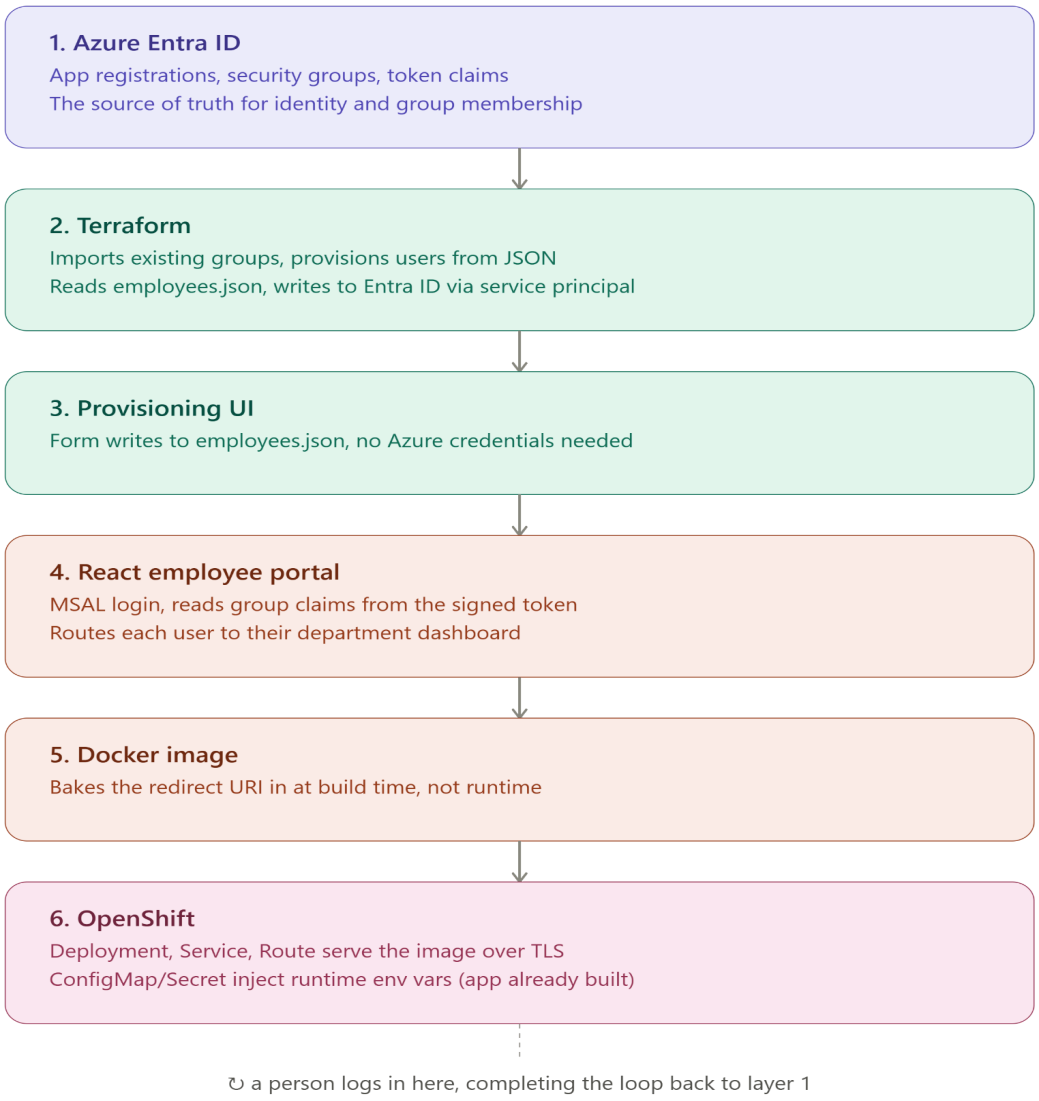
AUTOMATING

AUTOMATING THE PROJECT

Overview

With the manual RBAC setup proven working end-to-end, Phase 4 replaces the manual portal-clicking workflow from Phase 1 with Terraform — specifically, importing the existing Entra ID groups under Terraform's management and provisioning new employees from a structured data file instead of the Azure Portal UI.

This phase directly addresses the production concern raised earlier: manually creating users and assigning group membership through a web UI does not scale, is prone to human error, and creates configuration drift — if someone manually deletes or modifies a group outside of any tracked system, nothing detects it until something breaks downstream.



AUTOMATING THE PROJECT

Step 1: Download Terraform

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo

sudo dnf install terraform -y


terraform -version

**********Create Dedicated Terraform Project Folder******
mkdir -p ~/Azure-EntraID-OpenShift/terraform-entra-id
cd ~/Azure-EntraID-OpenShift/terraform-entra-id
```

Step 2: Authentication: Create a separate service principal for Terraform

A deliberate design decision here: Terraform was given its own App Registration (terraform-entraid-automation), entirely separate from the employee-portal application's identity. This keeps the blast radius contained — the credential that can create and delete users in the directory is not the same credential the web application uses to authenticate end users. If the application's secret were ever compromised, an attacker would gain login capability, not directory-write capability.

Since this is a backend, unattended identity with no user signing in interactively, the App Registration was created with no redirect URI — it authenticates via client credentials flow only.

This needs to be its own app registration in Azure - Separate from the employee portal- with permissions to manage users and groups

Go to Azure Portal —App registration – New registration

- Name: terraform-entraid-automation
- Supported account types: Single tenant
- Redirect URI: leave blank (this is a backend/service identity, not a user-facing app)

Click Register, then note down:

AUTOMATING THE PROJECT

- Application (client) ID
- Directory (tenant) ID (same as before, but confirm)

Then go to Certificates & secrets → New client secret, same as for employee-portal — copy the value immediately.

Step 3: Application Permissions, Not Delegated

This is a meaningful distinction worth calling out explicitly: the permissions granted (User.ReadWrite.All, Group.ReadWrite.All) were configured as **Application permissions**, not Delegated. Delegated permissions act on behalf of a signed-in user and are scoped to that user's own access; Application permissions allow the credential to act independently, with its own standing authorization, which is what an unattended automation pipeline requires. Both permission types additionally required explicit admin consent, since application-level write access to every user and group in the directory is a meaningfully privileged grant.

Go to your Terraform-entraid-automation App Registration → API permissions
Click + Add a permission → Microsoft Graph → Application permissions (not Delegated this time)
Search and add:

- User.ReadWrite.All
- Group.ReadWrite.All

This is important: application permissions require admin consent and grant Terraform the ability to act without a signed-in user — confirm both permissions show a green checkmark under "Status" after granting consent.

Step 4: Creating Terraform Files

- Create the project folder if you haven't

```
mkdir -p ~/Azure-EntraID-OpenShift/terraform-entra-id
cd ~/Azure-EntraID-OpenShift/terraform-entra-id
```

- Create providers.tf file

```
cat > providers.tf << 'EOF'
terraform {
  required_providers {
    azuread = {
      source  = "hashicorp/azuread"
```

AUTOMATING THE PROJECT

```
      version = "~> 2.47"
    }
  }
}

provider "azuread" {
  client_id     = var.client_id
  tenant_id     = var.tenant_id
  client_secret = var.client_secret
}
EOF
```

- Create variables.tf file

```
cat > variables.tf << 'EOF'
variable "client_id" {
  description = "Client ID of the terraform-entraid-automation service principal"
  type        = string
  sensitive   = true
}

variable "tenant_id" {
  description = "Azure AD tenant ID"
  type        = string
  sensitive   = true
}

variable "client_secret" {
  description = "Client secret for the terraform-entraid-automation service principal"
  type        = string
  sensitive   = true
}

variable "domain" {
  description = "Your Azure AD verified domain"
  type        = string
}
EOF
```

- Create groups.tf

```
cat > groups.tf << 'EOF'
resource "azuread_group" "portal_hr" {
  display_name      = "Portal-HR"
  description       = "Company HR personnels"
  security_enabled  = true
}

resource "azuread_group" "portal_it" {
```

AUTOMATING THE PROJECT

```
  display_name     = "Portal-IT"
  security_enabled = true
}

resource "azuread_group" "portal_engineering" {
  display_name     = "Portal-Engineering"
  security_enabled = true
}

resource "azuread_group" "portal_admins" {
  display_name     = "Portal-Admins"
  security_enabled = true
}
EOF
```

- Create .gitignore (so real secrets never get committed)

```
cat > .gitignore << 'EOF'
*.tfstate
*.tfstate.*
.terraform/
.terraform.lock.hcl
terraform.tfvars
crash.log
*.tfvars.json
EOF
```

- Create Terraform.tfvars with your real values - type this directly, do not paste it anywhere else

```
Vim terraform.tfvars

client_id     = ""
tenant_id     = ""
client_secret = "PASTE_YOUR_REAL_SECRET_HERE"
domain        = ""
```

- Terraform init

AUTOMATING THE PROJECT

```
Initializing provider plugins found in the configuration...
- Finding hashicorp/azuread versions matching "~> 2.47"...
- Installing hashicorp/azuread v2.53.1...
- Installed hashicorp/azuread v2.53.1 (signed by HashiCorp)

Initializing the backend...


Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

- Import all groups using Terraform:
- Because the four Portal-* groups already existed — created manually during Phase 1, with Object IDs already hardcoded into the React application's rbacConfig.js — Terraform was deliberately pointed at these existing groups via terraform import, rather than allowed to create fresh ones.

  This is an important operational pattern: importing brings real, already-running infrastructure under Terraform's management without disrupting it, whereas allowing Terraform to create new resources with the same names would generate entirely new Object IDs — silently breaking every part of the system that referenced the old ones, including the live application's authorization logic.

```
terraform import azuread_group.portal_hr <object-id>
terraform import azuread_group.portal_it <object-id>
terraform import azuread_group.portal_engineering <object-id>
terraform import azuread_group.portal_admins <object-id>
```

- Confirm all 4 are tracked

terraform state list

```
azuread_group.portal_admins
azuread_group.portal_engineering
azuread_group.portal_hr
azuread_group.portal_it
```

AUTOMATING THE PROJECT

Step 5: Verifying ZeroDrift Before Touching Anything

After import, the Terraform plan was run before any apply — a deliberate safety gate. The first plan revealed that the group resource definitions in code were missing the description field that existed on the real groups, and Terraform proposed overwriting those descriptions to match the incomplete code. Rather than applying that change, the Terraform configuration was corrected to match the real, existing state. A second Terraform plan then returned:

Terraform plan

```
azuread_group_member.employee_membership["Jordan.Finance"]: Refreshing state...
[id=40ff0000-f507-4fd6-9902-ba6710ee251b/member/9fd56d70-9f5d-4ca5-93e4-943e7973
1912]
azuread_group_member.employee_membership["Casey.Support"]: Refreshing state... [
id=cb8ffe78-d7b3-4769-98c6-95abc7fbf5be/member/1a11f6b5-6a51-4f09-8162-a452b8f2a
43c]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.
```

This is the correct sequence for adopting existing infrastructure into Terraform: import, plan, reconcile the code to reality, confirm zero drift — only then is it safe to begin making changes through Terraform going forward.

PHASE 5 — Provisioning Employees from Data, Not Clicks

The Core Mechanism

With groups safely under management, employee provisioning was rebuilt around a single JSON file (employees.json) describing each person

```
[
  { "first_name": "Jordan", "last_name": "Finance", "department": "HR", "status": "active" },
  { "first_name": "Casey", "last_name": "Support", "department": "IT", "status": "active" }
]
```

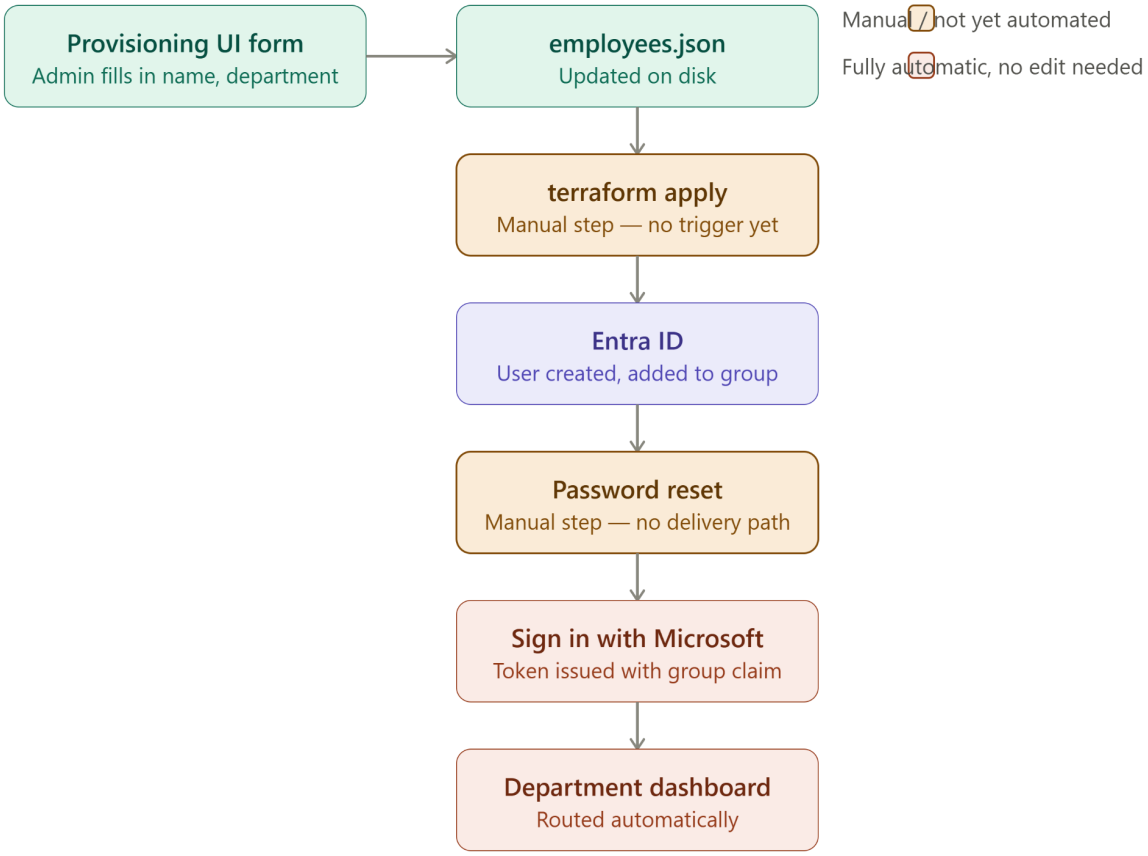
AUTOMATING THE PROJECT

Terraform reads this file, generates a stable per-employee key, and for every active entry: generates a strong random password, creates the Entra ID user account with force_password_change enabled, and adds them to the group matching their declared department — using a lookup map rather than requiring the JSON file to contain raw Object IDs. This keeps the data file human-readable and decoupled from Entra ID's internal identifiers.

```
locals {
  department_groups = {
    HR          = azuread_group.portal_hr.object_id
    IT          = azuread_group.portal_it.object_id
    ENGINEERING = azuread_group.portal_engineering.object_id
    ADMIN       = azuread_group.portal_admins.object_id
  }
}
```

AUTOMATING THE PROJECT

PHASE 6: UI INTERFACE



Overview

Small Express app, server-rendered HTML (no SPA framework needed), with a table reading current state from employees.json and a form below it for add/offboard actions — each action writing back to the file. This is also the simplest version to later wire into Git commit-on-submit, since it's just a backend file write either way

Step 1: Create the folder structure

```
mkdir -p ~/Azure-EntraID-OpenShift/provisioning-ui/views
mkdir -p ~/Azure-EntraID-OpenShift/provisioning-ui/public
cd ~/Azure-EntraID-OpenShift/provisioning-ui
```

AUTOMATING THE PROJECT

Step 2: Create package.json

```
{
  "name": "provisioning-ui",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "ejs": "^3.1.10"
  }
}
```

Step 3: Create server.js

Note: Please see the code in the git repo

Step 4: Create views/index.ejs

Note: Please see the code in the git repo

Step 5: Create public/style.css

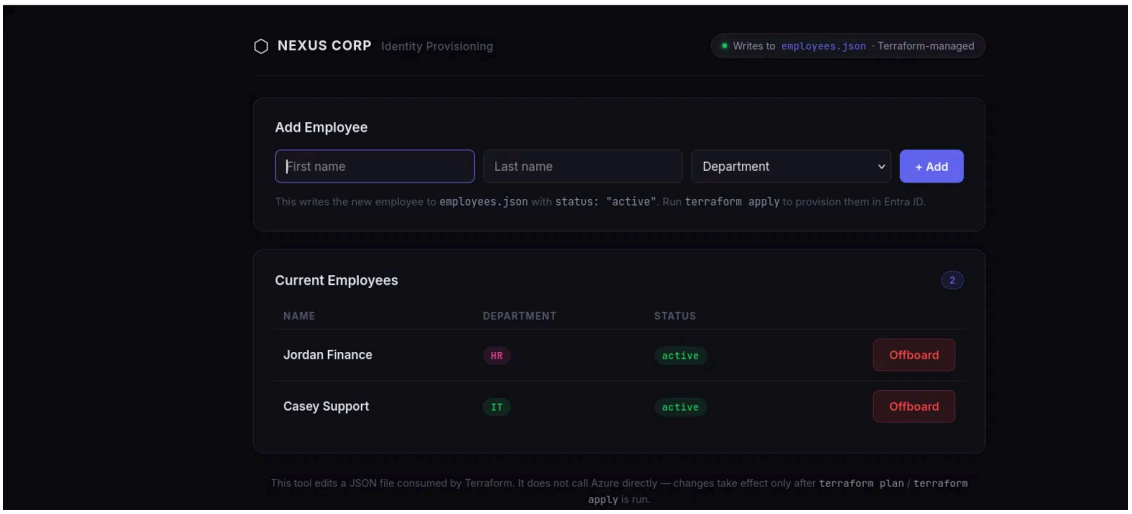
Note: Please see the code in the git repo

Step 6: Install dependencies

```
Npm install

******* Run it ********

Node server.js

********** You should see**************
Provisioning UI running at http://localhost:4000
Reading/writing: /home/[user]/Azure-EntraID-OpenShift/terraform-entra-id/employees.json
```

AUTOMATING THE PROJECT

- Access it



This is exactly right. 🎯

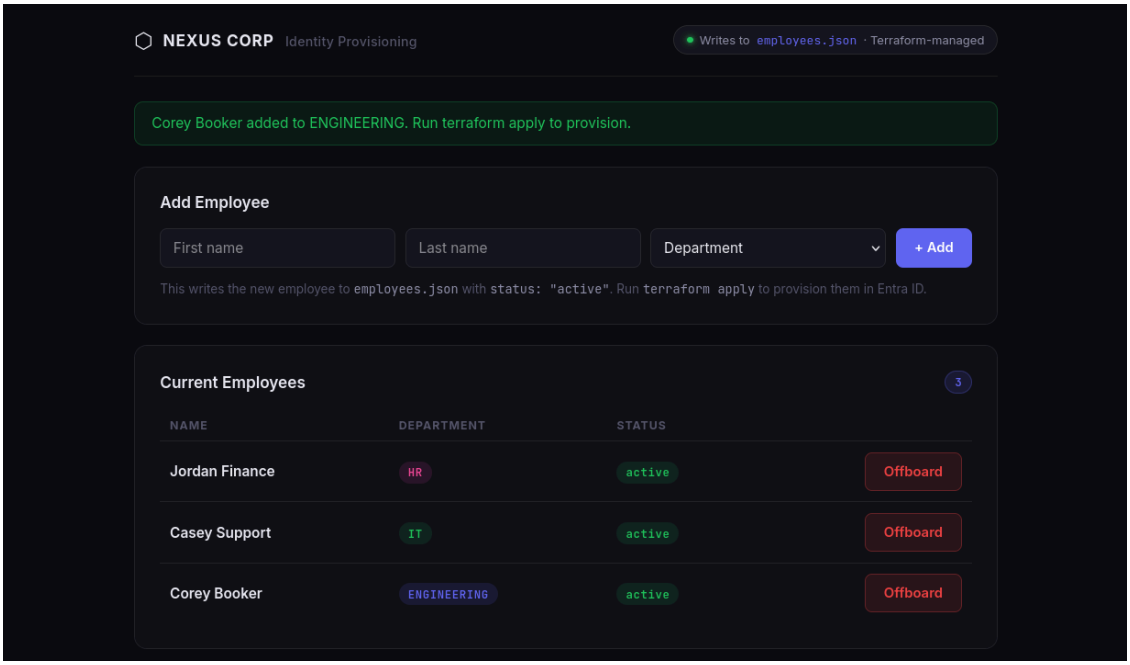
It's reading your real employees.json directly — both Jordan Finance (HR) and Casey Support (IT) show up correctly, with the right department color-coding and active status. This is genuinely a clean, professional-looking internal admin tool.

Let's prove the full loop now. **Add a new test employee through this form:**

1. Type a first name, last name
2. Pick a department from the dropdown
3. Click **+ Add**

***** Added User “Corey Booker” to the engineering dept*****

AUTOMATING THE PROJECT



Step 7: Success Message

Confirm the new row appears in the table

cat ~/Azure-EntraID-OpenShift/terraform-entra-id/employees.json

```
  },
  {
    "first_name": "Casey",
    "last_name": "Support",
    "department": "IT",
    "status": "active"
  },
  {
    "first_name": "Corey",
    "last_name": "Booker",
    "department": "ENGINEERING",
    "status": "active"
  }
]
```

Note: Web form → server.js writes file → employees.json (same file Terraform reads)

Now let's close the loop completely — run Terraform and watch it pick up this UI-created entry:

AUTOMATING THE PROJECT

cd ~/Azure-EntraID-OpenShift/terraform-entra-id
terraform plan

```
Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # azuread_group_member.employee_membership["Corey.Booker"] will be created
  + resource "azuread_group_member" "employee_membership" {
      + group_object_id  = "437e3327-910b-407f-82ef-b3a3287f8454"
      + id               = (known after apply)
      + member_object_id = (known after apply)
    }

  # azuread_user.employee["Corey.Booker"] will be created
  + resource "azuread_user" "employee" {
      + about_me                      = (known after apply)
      + account_enabled               = true
      + business_phones               = (known after apply)
```

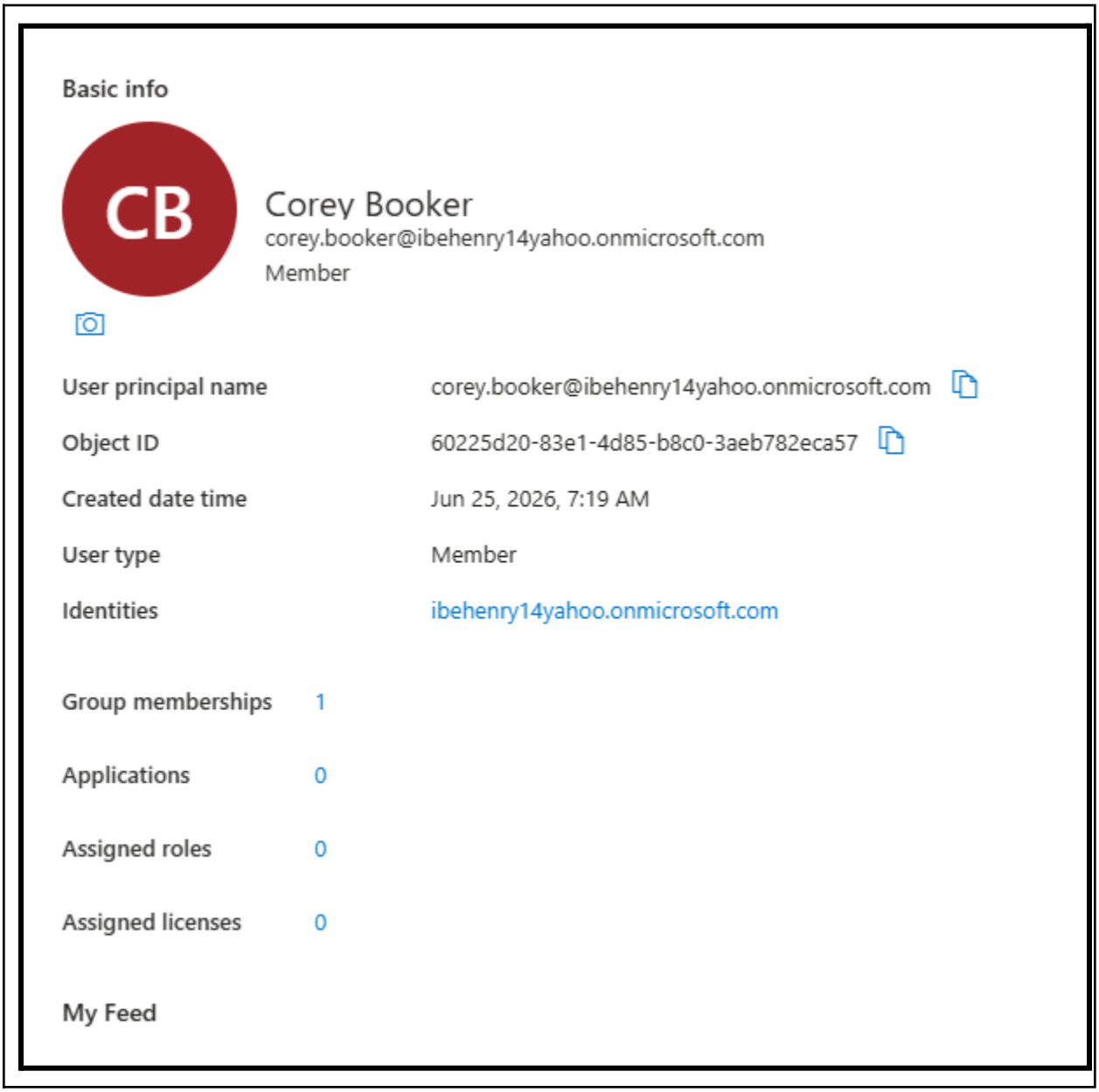
Terraform apply

Verification :

Step 1 — Confirm Corey exists in Entra ID and is in the right group

Quick check in Azure Portal: **Microsoft Entra ID** → **Users** → confirm Corey Booker appears, and **Groups** → **Portal-Engineering** → **Members** → confirm Corey's there.

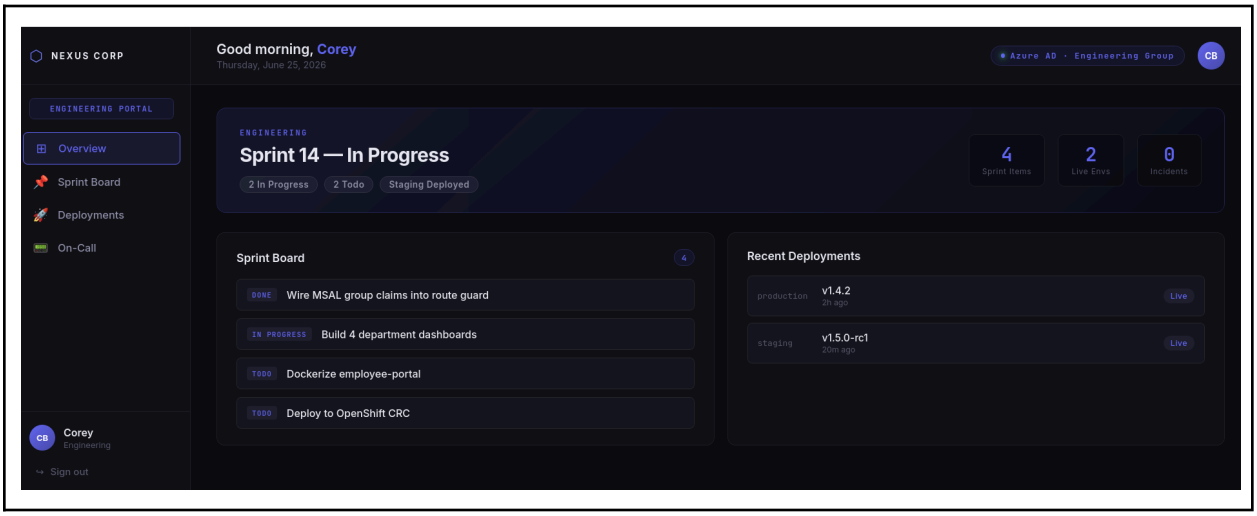
AUTOMATING THE PROJECT



Step 2 — The real payoff: log in to the employee portal as Corey

Since the account was created with force_password_change = true, you'd need Corey's auto-generated password to do this live, but Terraform doesn't print it by default (it's marked as sensitive). For testing purposes, you could manually reset Corey's password in the Azure Portal to log in and confirm they land on the **Engineering dashboard** (indigo theme) — proving that the entire chain from "fill out a form" to "a person can log in and see their correct department" is real.

AUTOMATING THE PROJECT



Known Limitations & Next Steps

This project intentionally scoped out several things a production system would require, in order to focus on demonstrating the core identity-as-code and RBAC patterns clearly. Naming these explicitly is part of the engineering discipline this project is meant to show — knowing what you didn't build, and why, is as important as what you did.

Terraform state is local, not remote. State currently lives only on the machine where terraform apply was run. There is no shared backend (Azure Storage, Terraform Cloud, etc.), no state locking, and no protection against two people running apply concurrently and corrupting state. A production setup would store state remotely from day one — this is the single most consequential gap if this were handed to a second engineer.

Authorization is enforced at the frontend only. The React route guard reads token claims and renders the appropriate dashboard, but nothing prevents a sufficiently motivated user from inspecting network requests or calling an API directly with a valid token and no group claim, if such an API existed. There currently is no backend API serving sensitive data, so this is a smaller risk than it would be in a fuller application — but the pattern, as built, is a UX/routing convenience, not a security boundary. A production version would validate the token and group claims server-side before returning any protected data.

New employee passwords require manual intervention. Terraform generates a strong random password and marks the account for forced change on first login, but there is no automated way to deliver that password to the actual new hire — an admin currently has to manually reset it in the Azure Portal to hand it off. A real onboarding flow would integrate with email delivery or a self-service first-login flow, rather than depending on someone copying a value out of Terraform's state.

AUTOMATING THE PROJECT

Offboarding is additive logic on the same resource type, not yet wired to automation triggers. The `status: "disabled"` field and the UI's Offboard button correctly flip a flag that Terraform can act on, but this still requires a human to run `terraform apply` afterward. There is no pipeline yet watching for that change and applying it automatically — that's the explicit next phase (Git-triggered CI/CD), not yet built.

No automated drift detection. If someone manually changes or deletes a group or user directly in Entra ID outside of Terraform, nothing currently notices until the next manual `terraform plan`. A production setup would run `terraform plan` on a schedule (e.g., nightly via CI) and alert on any detected drift, rather than relying on someone remembering to check.

No pre-commit secret scanning. The secrets-hygiene incident documented above was caught by manual review before a commit, not by tooling. A more mature setup would use something like `git-secrets` or `detect-secrets` as a pre-commit hook, so credential-shaped strings are blocked from being staged in the first place, rather than relying on a human catching it every time

Destructive actions have no approval gate. Right now, any change — additive or destructive — that reaches `terraform apply` takes effect immediately, with the same level of human review (a single person looking at a `plan` before typing `apply`). A production identity pipeline, especially one capable of disabling or deleting accounts, would require a second human approval step before a destructive plan is allowed to apply — particularly once this is wired into CI/CD, where the natural next failure mode is an automated pipeline applying a destructive change with no human in the loop at all.

Cleaning UP

SANITATION BEFORE COMMIT

```
cd ~/Azure-EntraID-OpenShift
pwd
ls
```

Let us run a full secrets audit,

```
grep -rn --include="*.tf" --include="*.yaml" --include="*.yml" --include="*.js" --include="*.json"
-E
"(client_secret|CLIENT_SECRET|password|tenant_id.*=.*[0-9a-f]{8}-|client_id.*=.*[0-9a-f]{8}-)"
. 2>/dev/null | grep -v node_modules | grep -v "YOUR_" | grep -v ".example"
```

Problem: The real Tenant ID, Client ID, and Client Secret are all hard-coded. That is a security issue.

Possible Solution: Rename the current file, for example, to be local and create a clean and committable template

Example:

```
cd ~/Azure-EntraID-OpenShift/employee-portal
mv openshift-deploy.yaml openshift-deploy.local.yaml
```

Add the local file to .gitignore

```
cd ~/Azure-EntraID-OpenShift/employee-portal
cat .gitignore
```

Step 2: Check if all sensitive files are ignored for commit

```
cd ~/Azure-EntraID-OpenShift
git status --ignored
```

```
employee-portal/.env
employee-portal/openshift-deploy.local.yaml
terraform-entra-id/terraform.tfvars
terraform-entra-id/terraform.tfstate
```

SANITATION BEFORE COMMIT

```
terraform-entra-id/terraform.tfstate.backup
```

Step 3: Audit the provisioning UI

This confirms the provisioning UI has no hardcoded credentials - It should only reference employees.json paths, nothing Azure -related directly.

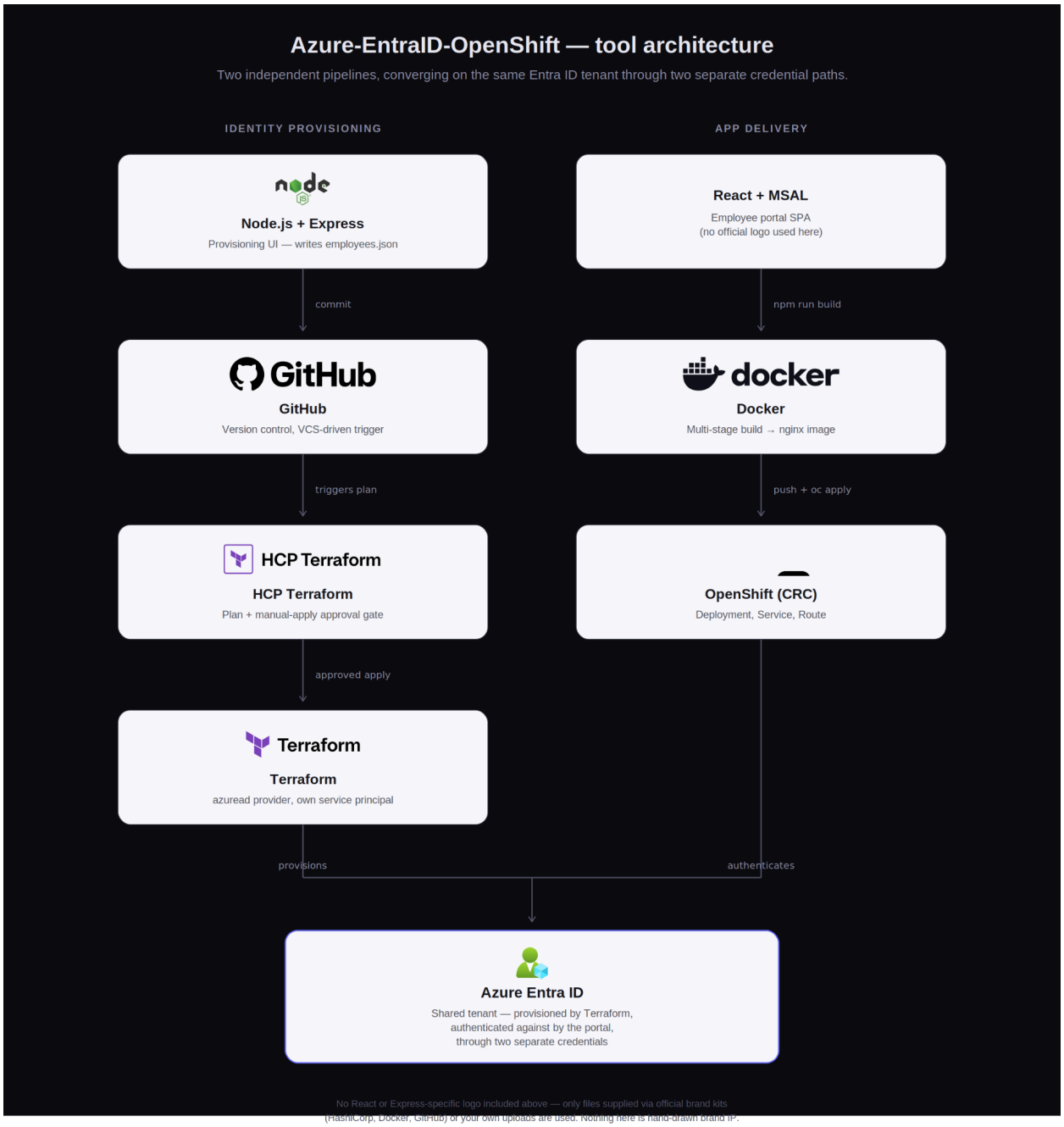
```
ls provisioning-ui/
cat provisioning-ui/server.js | grep -i -E "(secret|password|client_id|tenant)"
```

Then let's also do a final full-repo sanity sweep — one more grep pass across literally everything, including the provisioning-ui folder we just cleared, to catch anything we might have missed:

```
cd ~/Azure-EntraID-OpenShift
grep -rn -E "[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}" --include="*.tf"
--include="*.yaml" --include="*.yml" --include="*.js" --include="*.json" --include="*.ejs" .
2>/dev/null | grep -v node_modules | grep -v ".terraform" | grep -v "YOUR_"
```

How to run the project

How to Run This Project



How to Run This Project

How to Run This Project

A practical setup guide, from a fresh clone to a working login. This assumes you already have your own Azure tenant and basic familiarity with oc/Terraform/npm.

Prerequisites

- An Azure AD (Entra ID) tenant you can administer
- Node.js 18+
- Terraform
- Docker or Podman
- An OpenShift cluster (CRC locally, or a Developer Sandbox)
- Git

1. Clone the repo

```
git clone https://github.com/henry-ibe/Azure-EntraID-OpenShift.git
cd Azure-EntraID-OpenShift
```

2. Set up Azure Entra ID

App Registration #1 — the web app

- Azure Portal → App registrations → New registration
- Name: employee-portal
- Platform: Single-page application
- Redirect URI: http://localhost:3000
- API permissions → Microsoft Graph → Delegated → User.Read → grant admin consent
- Authentication → enable ID tokens + Access tokens
- Note: Client ID, Tenant ID

App Registration #2 — Terraform's service principal

- New registration, name: terraform-entraid-automation
- No redirect URI (backend identity)
- Certificates & secrets → new client secret → copy the value

How to Run This Project

- API permissions → Microsoft Graph → **Application** permissions → User.ReadWrite.All, Group.ReadWrite.All → grant admin consent
- Note: Client ID, Client Secret

Security groups
Create 4 groups (Security, Assigned membership): Portal-HR, Portal-IT, Portal-Engineering, Portal-Admins. Note each Object ID.

Token configuration (on employee-portal)

- Token configuration → Add groups claim → Security groups → ID token: Group ID

3. Configure and run the React app

```
cd employee-portal
cp .env.example .env
```

Edit .env:

```
REACT_APP_AZURE_CLIENT_ID=<employee-portal Client ID>
REACT_APP_AZURE_TENANT_ID=<your Tenant ID>
REACT_APP_REDIRECT_URI=http://localhost:3000
```

Edit src/rbacConfig.js — replace the 4 placeholder GUIDs with your real group Object IDs.

```
npm install --legacy-peer-deps
npm install ajv@^8.0.0 --legacy-peer-deps   # fixes a known react-scripts/ajv conflict
HTTPS=true npm start
```

Visit https://localhost:3000, accept the self-signed cert warning, and sign in.

4. Set up Terraform

```
cd ../terraform-entra-id
```

How to Run This Project

```
cp terraform.tfvars.example terraform.tfvars
```

Edit terraform.tfvars with your service principal's real values (Client ID, Tenant ID, Client Secret, domain).

```
terraform init
```

If your 4 groups already exist (created manually above), import them so Terraform doesn't recreate them with new Object IDs:

```
terraform import azuread_group.portal_hr <hr-group-object-id>
terraform import azuread_group.portal_it <it-group-object-id>
terraform import azuread_group.portal_engineering <engineering-group-object-id>
terraform import azuread_group.portal_admins <admin-group-object-id>
```

```
terraform plan   # should report "No changes" once groups.tf matches reality
```

5. Provision employees

Edit employees.json directly, or use the UI:

```
cd ../provisioning-ui
npm install
node server.js
```

Visit http://localhost:4000 → add an employee via the form. This writes to ../terraform-entra-id/employees.json.

Then apply it:

How to Run This Project

```
cd ../terraform-entra-id
terraform plan
terraform apply
```

Each active entry gets a real Entra ID user, a generated password (force_password_change = true), and group membership matching their department

6. Deploy to OpenShift

```
cd ../employee-portal
docker build -t employee-portal:latest .
docker tag employee-portal:latest <your-registry>/employee-portal:latest
docker push <your-registry>/employee-portal:latest
```

```
cp openshift-deploy.yaml openshift-deploy.local.yaml
```

Edit openshift-deploy.local.yaml — replace YOUR_TENANT_ID, YOUR_CLIENT_ID, YOUR_CLIENT_SECRET, YOUR_REGISTRY, YOUR_CLUSTER_DOMAIN

```
oc login <your-cluster-api-url>
oc apply -f openshift-deploy.local.yaml
oc get pods -n employee-portal
oc get route employee-portal -n employee-portal
```

Add the resulting Route URL as an additional redirect URI on the employee-portal App Registration in Azure (SPA platform). **Note:** since REACT_APP_* values are baked in at build time, the redirect URI in .env at build time must already match the Route URL — rebuild and redeploy if it doesn't.

7. Log in and verify RBAC

Visit the Route URL → Sign in with Microsoft → use one of your provisioned employees' credentials (reset their password in Azure Portal first if needed, since force_password_change blocks login with the Terraform-generated one until reset).

How to Run This Project

Expected result: each user lands on the dashboard matching their group (HR/IT/Engineering/Admin). A user in no group sees the **Access Restricted** page instead.

A note on /etc/hosts and accessing the cluster remotely

If you're running CRC on one machine and accessing it from a browser on a **different** machine (for example, SSH'd into a homelab server from your laptop), you will hit connectivity problems that look like DNS failures but aren't.

Why this happens: CRC's router only binds to 127.0.0.1 on the machine it's running on — it does not listen on that machine's LAN IP by default. This is a deliberate local-only default, not a misconfiguration.

The fix — SSH tunnel + local hosts entries:

On the machine running CRC, confirm the route hostname:

```
oc get route employee-portal -n employee-portal
```

On your **local machine** (the one with the browser), add entries to /etc/hosts pointing the cluster's hostnames at 127.0.0.1:

```
127.0.0.1  employee-portal.apps-crc.testing
127.0.0.1  console-openshift-console.apps-crc.testing
127.0.0.1  oauth-openshift.apps-crc.testing
127.0.0.1  api.crc.testing
```

Then open an SSH tunnel from your local machine to the CRC host, forwarding the cluster's HTTPS port to a local port (443 itself usually requires root, so a higher port is simpler):

```
ssh -L 8443:127.0.0.1:443 user@<crc-host-ip>
```

Keep that terminal open, then access everything with the port explicitly in the URL:

How to Run This Project

https://employee-portal.apps-crc.testing:8443

Two things that will trip you up if you skip the port:

- Typing the URL without:8443 silently falls back to port 443, which isn't tunneled, and looks identical to a DNS failure in the browser.
- If you previously configured Entra ID as an OpenShift cluster login provider (a separate, unrelated integration), the web console's OAuth login flow generates redirect URLs using the cluster's *real* hostname/port — not your tunnel's port — and will break in a way no /etc/hosts edit can fix. If you hit this, skip the web console entirely and rely on oc get pods / oc get route for verification; it isn't required for anything in this guide.

If you're accessing the browser on the *same* machine running CRC, none of this applies — CRC's own DNS/hosts setup handles it automatically, and you can skip this section entirely.

Common gotchas

- **MSAL crypto error** → must use HTTPS, not HTTP, for local dev
- **CRC route unreachable from another machine** → CRC's router binds to 127.0.0.1 only; tunnel with ssh -L 8443:127.0.0.1:443 user@host
- **Login redirects to the wrong URL** → REACT_APP_REDIRECT_URI was wrong *at build time*; fix .env and rebuild the image, don't rely on ConfigMap/Secret to fix it post-build
- **terraform import 403 error** → Application permissions not actually granted consent yet; re-check and re-grant in Azure

Additional Troubleshooting

A few smaller issues you may hit depending on your environment, in rough order of likelihood.

Don't run npm audit fix --force
If npm install prints vulnerability warnings, ignore them — they're almost always in dev-only build tooling (webpack, etc.) and don't affect runtime. Running npm audit fix --force can silently downgrade react-scripts to a broken version (we hit this directly — it corrupted react-scripts to

How to Run This Project

"^0.0.0" in package.json, after which nothing would install or run). If this happens to you: delete node_modules and package-lock.json, fix the react-scripts version back to 5.0.1 in package.json by hand, then reinstall.

Local docker run networking may not reflect real connectivity

If you're using Podman (aliased as docker via podman-docker), rootless networking can make docker run -p 8080:8080 hang or silently fail to forward traffic on localhost, even though the container is healthy (docker exec <container> curl localhost:8080 will confirm it's serving fine internally). This is a local testing inconvenience, not a sign your image is broken — OpenShift manages networking completely differently via Services and Routes, so don't spend much time chasing this; the real test is the actual cluster deployment.

Docker Hub push errors with Podman

If docker push fails with Failed to parse the authentication scope from the given challenge, this is a known Podman quirk, not a real auth failure — push again using the fully-qualified registry path explicitly:

```
docker tag employee-portal:latest docker.io/<your-username>/employee-portal:latest
docker push docker.io/<your-username>/employee-portal:latest
```

Also make sure your Docker Hub access token has **Read & Write** scope, not just Read-only — a read-only token will fail on push with an access-denied error that looks unrelated to scope.

If curl works but your browser doesn't, suspect machine topology first

If a command-line curl to a URL succeeds but the same URL fails in your browser, don't immediately chase browser settings (DNS-over-HTTPS, proxy config, cache). First confirm both are actually running on the *same* machine and using the *same* network path — curl run inside an SSH session reaches the remote host directly, while a browser on your physical machine does not, even if both seem to be "on the same network." This single mismatch can cost significant time if you debug it as a browser configuration problem instead of a topology one.

Git Triggered CI/CD

GitOps-Driven Identity Provisioning with Terraform Cloud

Overview

This phase closes the last manual gap in the system: nothing currently triggers terraform apply when employees.json changes — a human has to remember to run it. This guide connects the repo to Terraform Cloud so that a commit automatically triggers a plan, with a human approval step before that plan can apply.

GitHub Actions is deliberately kept out of the execution path. CI runners are shared, broadly-scoped environments — giving one the credential to mutate Entra ID would put every workflow in the repo inside that credential's blast radius. Terraform Cloud instead owns the state, the credentials, and the approval gate, with its own audit trail of every plan and every approver. GitHub's only job is to signal that something changed; Terraform Cloud owns deciding what to do about it.

Prerequisites: a fork or clone of the Azure-EntraID-OpenShift repo, admin access to your own Azure AD tenant, and the Terraform-side setup from GETTING_STARTED.md already completed (service principal created, terraform.tfvars.example values in hand). You don't need an existing Terraform Cloud account — this guide creates one.

Step 1 — Create your Terraform Cloud account

Go to app.terraform.io and sign up (free tier, no credit card required). Use whatever email you'd like — this is separate from your Azure account entirely.

Step 2 — Create an organization

Terraform Cloud requires an "organization" as the top-level container for your workspaces. Pick any name you like — it's just a namespace, not visible to anyone unless you invite them. Throughout the rest of this guide, replace <your-org-name> with whatever you chose.

- Click "Get started with Terraform"
- Click "Go to HCP Terraform"

GitOps-Driven Identity Provisioning with Terraform Cloud

- Enter your organization name and click **Create organization**

Step 3 — Create a workspace

- Click **New workspace** (or "get started with a workspace" if this is your first one)
- Choose **Version Control Workflow**
- Choose **GitHub** as the VCS provider
- Authorize the Terraform Cloud GitHub App if prompted, and grant it access to your fork/clone of the repo specifically — not every repo on your account. Least privilege applies to the VCS connection just as much as to the Azure service principal.
- Select your copy of Azure-EntraID-OpenShift from the repository list. If it doesn't appear, the GitHub App authorization was scoped to the wrong repo or account — go back into GitHub's App settings (github.com/settings/installations) and add it there.
- Give the workspace a name — anything descriptive works, e.g. entra-id-provisioning. Replace <your-workspace-name> with whatever you choose from here on.

Step 4 — Configure settings

This is the step that trips people up, because Terraform Cloud assumes .tf files live at the repo root by default. They don't — this repo has three top-level folders, and Terraform only owns one of them.

- **Terraform Working Directory** → set to terraform-entra-id. Without this, the first run fails with "No configuration files found" even though the repo connection itself is correct.
- **VCS branch** → main (or whichever branch you commit employees.json changes to).
- **Automatic run triggering** → choose "Only trigger runs when files in specified paths change" and add the path terraform-entra-id/*. Without this filter, every commit to employee-portal/ or provisioning-ui/ — which have nothing to do with Terraform — queues a needless plan.

Step 5 — Add workspace variables

Add the four variables Terraform expects, matching variables.tf. Use your own service principal's real values here — not the placeholders from terraform.tfvars.example.

Key	Value	Category	Sensitive?

GitOps-Driven Identity Provisioning with Terraform Cloud

client_id	your terraform-entraid-automation app's Client ID	Terraform variable	Yes
tenant_id	your Azure AD Tenant ID	Terraform variable	Yes
client_secret	your service principal's Client Secret	Terraform variable	**Yes**
domain	your verified domain, e.g. yourtenant.onmicrosoft.com	Terraform variable	No

Set the category to **Terraform Variables**, not Environment Variables — these map to the variable blocks in variables.tf, not to shell env vars the provider reads implicitly. Checking **Sensitive** write-protects the value after save: it disappears from the UI and the API response, and won't be echoed into run logs.

Click **Create workspace** (or **Save** if the workspace already exists from Step 3).

Step 6 — Point the config at your workspace (migrate state)

If your fork's Terraform config has no cloud block yet, it's still using local state — the exact limitation called out in the main README ("Terraform state is local. No shared backend, no state locking"). Closing that gap is the actual point of this phase.

Add to terraform-entra-id/providers.tf, using **your** org and workspace names from Steps 2 and 3:

```
terraform {
  cloud {
    organization = "<your-org-name>"
    workspaces {
      name = "<your-workspace-name>"
    }
  }
}
```

Then, from your own machine (not CI):

GitOps-Driven Identity Provisioning with Terraform Cloud

```
terraform login          # opens a browser, generates an API token, stores it locally
cd terraform-entra-id
terraform init           # detects the new cloud block and offers to migrate local state
```

Confirm the migration when prompted. From this point on, terraform.tfstate lives in Terraform Cloud, not on disk — the local file becomes a stale artifact and can be deleted (it's already gitignored, so this doesn't touch the repo).

If you're starting fresh with no prior local state, terraform init will just connect to the empty remote workspace — nothing to migrate, nothing to do differently.

Step 7 — Set the apply method (the approval gate)

This is the control the whole phase exists to add. By default, a new VCS-driven workspace is set to **Auto apply**, which would defeat the purpose — a commit to employees.json would immediately create or delete real Entra ID accounts with no review.

In your workspace: **Settings → General → Apply Method → Manual apply.**

With this set, a plan runs automatically on every qualifying commit, but it stops and waits in **Needs Confirmation** state until a human reviews the plan output and clicks **Confirm & Apply**. Anyone with write access to the workspace can be an approver — for a solo project that's just you, but the mechanism is identical to what a team would use for a second-approver policy.

Step 8 — Test the full loop

1. Add or offboard an employee, either by editing employees.json directly or through provisioning-ui (node server.js → http://localhost:4000).
2. Commit and push the change to the branch you configured in Step 4.
3. In your Terraform Cloud workspace, a new run should appear within a few seconds — this confirms the path filter and VCS webhook are wired correctly.
4. Open the run and read the plan diff. Verify it matches what you expect (e.g. one azuread_user add, one azuread_group_member add — not more).
5. Click **Confirm & Apply**. Terraform Cloud applies using the credentials stored in your workspace variables — your local machine and GitHub Actions never touch the Azure service principal at any point in this flow.

GitOps-Driven Identity Provisioning with Terraform Cloud

If everything is wired correctly, the workspace's resource list should end up matching what's declared across groups.tf and users.tf: 4 groups, plus one azuread_user, one random_password, and one azuread_group_member per active employee in employees.json.

This closes the loop described in the overview: employees.json changes on your branch → Terraform Cloud plans automatically → a human approves → Entra ID is mutated, with a full audit trail of who approved what and when, visible in the workspace's run history.

Common gotchas

- **"No configuration files found" on the first run** → the working directory wasn't set to terraform-entra-id in Step 4. Fix it in Settings → General and re-queue the run.
- **Every unrelated commit triggers a plan** → the path filter from Step 4 wasn't saved, or was set to /* instead of terraform-entra-id/*.
- **Client secret visible in a run log** → the Sensitive checkbox wasn't checked when the variable was created. Delete the variable and re-add it with Sensitive enabled — editing an existing non-sensitive variable to sensitive after the fact doesn't retroactively scrub old logs.
- **terraform init doesn't offer to migrate state** → the cloud block is missing or malformed in providers.tf, the org/workspace names don't match what you created in Steps 2–3, or you're not logged in (terraform login) with a token that has access to the org.
- **A stale local terraform.tfvars causes confusing plan diffs** → once the cloud block is present, Terraform Cloud's workspace variables are authoritative. A leftover local terraform.tfvars is simply ignored for cloud-backed runs, which is easy to forget if you're used to local apply.
- **Run stuck in "Needs Confirmation" with no notification** → Terraform Cloud's free tier doesn't email on every run by default; check workspace notification settings under Settings → Notifications, or get in the habit of checking the Runs tab after a commit.
- **GitHub App can't see the repo** → if you forked the project, the app needs to be authorized against your account/org, not the original henry-ibe/Azure-EntraID-OpenShift. Check github.com/settings/installations.